

M. S. Ramaiah University of Applied Sciences
School of Social Sciences



Department of Data Sciences and Analytics

From Behavioural Drift to Churn: A Temporal Framework for High-Value Telecom Customers with Optimised Retention Allocation.

Submitted by:

Vijay Rangaswamy | 23SSDS415020

May-2026

SCHOOL OF SOCIAL SCIENCES

M. S. RAMAIAH UNIVERSITY OF APPLIED SCIENCES

Bengaluru – 560054

Contents

Chapter	Title	Page
1	Introduction	5-7
2	Literature Review	8-10
3	Methodology	11-19
4	Discussion – Results and Analysis	20-33
5	Conclusion	34-36
	Contribution	37-38
References		39
Annexure	Code Snippets and Variable Glossary	40-41

Chapter – 1

Introduction

1.1 Background and Context

We live in an era where the telecommunications industry is transforming faster than almost any other sector. Mobile connectivity has moved from being a luxury to an everyday necessity, and the companies that provide it are locked in an intensely competitive battle for subscribers. In India, this competition reached a fever pitch after the mid-2010s when new, aggressively priced operators entered the market, slashing Average Revenue Per User (ARPU) while simultaneously raising the bar for network quality, data speeds, and customer experience. Against this backdrop, keeping existing customers has become just as important, if not more so, than winning new ones. This is particularly true in the prepaid segment, where there are no contracts, no exit fees, and no switching costs. A prepaid customer can simply stop recharging with one operator and move to another, often within hours, with zero friction. That means the only early warning signs an operator has are behavioural signals hidden in the data: a customer recharging less frequently, using fewer minutes, downloading less data. These quiet signals, if read correctly, can be the difference between retaining a valuable subscriber and losing them forever.

The financial stakes of getting this wrong are enormous. Industry figures consistently show that annual churn rates in telecom hover between 15% and 25%, meaning that in any given year, roughly one in five customers leaves. What makes this especially painful is the asymmetry in costs: acquiring a new customer costs anywhere from three to five times more than retaining an existing one. When we narrow the focus to high-value customers, those who recharge at the highest levels and drive a disproportionate share of revenue, even a small improvement in retention rates translates into meaningful revenue protection. These are the customers that every operator can least afford to lose, and yet they receive no more targeted attention than the average subscriber under traditional, undifferentiated retention approaches.

1.2 Problem Statement

This dissertation tackles a very specific and practically important question: given four months of detailed behavioural data for high-value prepaid subscribers, is it possible to predict, with enough confidence and enough lead time, which of those customers will churn in the following month? And, critically, once we have identified the customers most at risk, how do we decide where to spend a finite retention budget? These two questions, prediction and action, are both addressed in this work. The first is answered through a machine learning modelling pipeline; the second through a supply chain-inspired resource allocation framework that translates probability scores into concrete, prioritised intervention decisions.

1.3 Objectives

To keep our work focused and measurable, we set out the following specific objectives at the outset of this project:

- To establish a rigorous, usage-based operational definition of churn appropriate for the prepaid telecom context.
- To identify and extract high-value customers using quantitative recharge-based criteria (70th percentile, ₹478).
- To engineer temporal behavioural features capturing month-on-month changes in usage, recharge, and engagement.
- To build and compare multiple machine learning classifiers for churn prediction, optimised for sensitivity under class imbalance.
- To develop a three-tier risk and value segmentation framework with a budget-constrained SCM-based resource allocation model.
- To translate model predictions into actionable business recommendations.

1.4 Three-Phase Implementation Framework

Our work is organised into three connected phases, each building on the last. Phase 1, Problem Understanding, involved getting to grips with the data, defining what churn actually means in a prepaid context, and building the conceptual three-phase customer lifecycle model that underpins everything else. This model divides the observation window into the Good Phase (Months 6–7), when the customer is stable and active; the Action Phase (Month 8), when behavioural deterioration begins and early warning signals appear; and the Churn Phase (Month 9), when the customer has fully disengaged. Phase 2, Machine Learning, translated that conceptual framework into a complete data science pipeline: preprocessing, feature engineering, model training, hyperparameter tuning, and rigorous evaluation across five model configurations. Phase 3, SCM-Based Optimisation, took the probability scores produced by the best model and fed them into a supply chain-inspired framework that assigns each at-risk customer to an intervention priority tier and allocates the retention budget accordingly.

This three-phase structure is used consistently throughout the study. All feature engineering, model building, and analysis steps are aligned with this framework, where Months 6 and 7 represent normal behaviour, Month 8 captures any behavioural shift, and Month 9 is used to determine whether the customer has churned.

1.5 Scope and Delimitations

It is important to be clear about what this study does and does not cover. The entire analysis is built on data from a single prepaid telecom operator within one Indian telecom circle (Circle ID 109), over a four-month window from June to September. The high-value cohort is defined using a ₹478 average monthly recharge threshold (70th percentile), which yields 30,001 subscribers for analysis. We do not examine postpaid churn, we do not compare across operators, and we do not attempt to build a real-time or streaming prediction system. These are conscious delimitations that sharpen the focus of the study rather than gaps, each would constitute a meaningful extension for future research.

Chapter – 2

Literature Review

2.1 Churn Prediction in Telecommunications

The academic literature on customer churn prediction is now more than two decades old, and the telecom sector has been at the centre of it almost from the beginning. Early work established that predicting churn is, at its core, a binary classification problem: given a set of features describing a customer's recent behaviour, will they leave or stay? Burez and Van den Poel (2009) made an important contribution by formally characterising the class imbalance problem that plagues most real-world churn datasets, the fact that churners are always in the minority, sometimes as few as 5–10% of the base. Their work showed that standard accuracy metrics are deeply misleading in this setting: a naïve model that predicts 'no churn' for every single customer would achieve 90%+ accuracy while being completely useless. They proposed cost-sensitive sampling strategies and probability threshold adjustment as practical remedies, both of which we implement in this dissertation. Verbeke et al. (2012) pushed the field further by reframing churn prediction as a profit maximisation problem rather than an accuracy maximisation problem, demonstrating that the most valuable models are those that correctly identify high-revenue churners, even at the cost of some additional false positives. This insight directly shapes our revenue-at-risk quantification and SCM allocation approach in Phase 3.

Looking across the broader literature, Random Forest and Gradient Boosting consistently emerge as the top-performing model families for telecom churn prediction. The reason is not hard to understand: both are ensemble methods that combine many weak learners into a strong one, which makes them naturally robust to the noisy, non-linear, high-dimensional feature spaces that characterise telecom datasets. Decision Trees sit at the other end of the interpretability-performance spectrum, they produce human-readable rules, but their predictive ceiling is significantly lower, especially on imbalanced data. Logistic Regression occupies an interesting middle ground: not the most powerful model, but one of the most practically deployed because its probability outputs are well-calibrated and easy to integrate into downstream business processes. We include all three families in our empirical comparison.

2.2 Temporal and Behavioural Modelling

One of the most persistent blind spots in the churn prediction literature is the almost universal reliance on static, cross-sectional data, a single month's worth of features used to predict whether a customer will churn in the following period. This approach treats every observation as independent and ignores the trajectory of the customer's behaviour over time. But churn rarely happens suddenly. In the real world, customers typically disengage gradually: they start recharging a little less, calling a little less, using a little less data, often over a period of weeks or months before they finally stop altogether. A model that only sees a snapshot misses this trajectory entirely. Temporal behavioural modelling addresses this by analysing change vectors across multiple consecutive periods rather than static point-in-time features. Studies employing longitudinal data have consistently found that month-over-month ARPU differentials, declining recharge frequency trends, and diminishing data consumption trajectories provide substantially greater predictive signal than any individual monthly snapshot. Our delta feature engineering approach, computing the difference between August activity and the June, July baseline, is grounded directly in this body of evidence. The three-phase lifecycle model operationalises this insight by explicitly naming the Action Phase as the critical intervention window: the period when deterioration is visible in the data but the customer has not yet fully churned.

2.3 High-Value Customer Segmentation

Not all customers are created equal, and a sound retention strategy should reflect that fact. The economics of customer lifetime value (CLV) have long argued that the resources devoted to retaining a customer should be proportional to the value that customer represents. In prepaid telecom, where there are no bills or contracts to anchor value estimates, recharge behaviour is the best available proxy: how much does the customer top up, how often, and how large are their individual recharges? The 70th percentile threshold we use to define high-value customers (₹478 average monthly recharge) is consistent with established industry practice. The economic logic is stark: this cohort of roughly 30% of subscribers typically contributes 60–70% of total prepaid revenue. Losing even a fraction of these customers has a revenue impact far disproportionate to their share of the subscriber base, which is precisely why they deserve a dedicated, data-driven retention effort.

2.4 Supply Chain Management Applied to Customer Retention

One of the more creative intellectual moves in this dissertation is the translation of supply chain management (SCM) thinking into the domain of customer retention. At first glance, the two might seem unrelated, but the structural parallel is surprisingly tight. In SCM, the central challenge is managing disruption risk: a key supplier might fail, a logistics route might become unavailable, demand might spike or collapse. Managers respond by tiering their suppliers by criticality, holding safety stock in proportion to disruption cost, and allocating mitigation resources where the expected loss is highest. In telecom retention, the same logic applies. Customer churn is a demand disruption event, a sudden, often irreversible reduction in the operator's revenue base. Churn probability scores are the demand disruption risk indices. Customer revenue contribution is the supply criticality measure. The retention budget is the mitigation resource pool. And the intervention priority framework, Immediate Action for High Risk + High Value, Targeted Action for High Risk + Low Value, No Action for the rest, maps directly onto the ABC inventory classification and tiered supplier risk protocols that have been standard in SCM for decades (Chopra and Meindl, 2015). We are not aware of prior work that applies this analogy as explicitly and operationally as we do here. Here, supply chain concepts are not used in a literal sense, but as a way to think about how to allocate limited resources. Just like in supply chains, where important suppliers are prioritised, we prioritise high-risk and high-value customers for retention.

2.5 Machine Learning Models

Before we describe our own models, it is worth briefly reviewing the methodological toolkit we draw on. Logistic Regression with Recursive Feature Elimination (RFE) provides a principled way to winnow a large feature space down to the variables with the most predictive signal, while VIF analysis guards against multicollinearity, a particular risk in temporal datasets where the same metric measured in adjacent months will inevitably be correlated with itself. Principal Component Analysis (PCA) takes this further by constructing entirely new, orthogonal features (principal components) that summarise the variance in the original space. With 168 features in our dataset, many of them correlated across months, PCA is an important tool for making linear models well-behaved. Random Forest sidesteps multicollinearity almost entirely through its tree-based structure, and its out-of-bag scoring mechanism provides an internal validation estimate without requiring a separate holdout partition (Breiman, 2001). Finally, probability threshold optimisation, setting the classification cutoff to 0.10 rather than the conventional 0.50, is a straightforward but powerful technique for rebalancing the sensitivity-specificity trade-off in favour of catching churners, which is the business priority in our setting (Fawcett, 2006).

Chapter – 3

Methodology

3.1 Dataset Description

The foundation of this entire analysis is a rich, large-scale prepaid telecom dataset sourced from a public GitHub repository. It contains complete monthly behavioural records for 99,999 subscribers across four consecutive months, June, July, August, and September, for a single operator in Circle ID 109, one of India's major telecom circles. With 226 variables per customer, the dataset is unusually comprehensive, covering every meaningful dimension of prepaid subscriber behaviour: how many calls they made and received, how much data they used, how often and how much they recharged, whether they used roaming services, which types of service plans they subscribed to, and how long they had been on the network. This richness is precisely what makes it possible to construct meaningful temporal features and to build prediction models with strong discriminative power.

Parameter	Value
Total Subscribers	99,999
Total Variables	226
Telecom Circle	109 (India)
Observation Period	June – September (Months 6–9)
High-Value Cohort (post-filter)	30,001
Churn Rate (High-Value Cohort)	8.14% (2,441 churners)
Final Features (after preprocessing)	168
Training Set	18,365 records (70%)
Test Set	7,871 records (30%)
Modelling Records (after outlier treatment)	26,236

3.2 Column Taxonomy

Before building the model, the dataset was cleaned and prepared to ensure consistency and reliability. This included handling missing values, imputing appropriate values based on feature type, and removing unnecessary features. The 226 variables are organised according to a consistent and logical naming convention that, once understood, makes the dataset very navigable. The numeric suffixes 6, 7, 8, and 9 refer to the calendar months June through September respectively. The alphabetic prefixes encode the nature of the measurement: ARPU for Average Revenue Per User, MOU for Minutes of Usage, RECH for Recharge, OG and IC for Outgoing and Incoming call directions, LOC and STD for local and long-distance calls, ROAM for roaming activity, VOL for data volume in megabytes, 2G and 3G for network generations, SACHET for short-validity service schemes, VBC for Volume-Based Cost (the per-unit charge incurred when no active plan is in place), and AON for Age on Network in days. Understanding this taxonomy is essential for interpreting the model results and feature importance rankings discussed in Chapter 4. The full classification was validated programmatically. A missing value analysis was performed to understand the extent and pattern of missing data across all features.

Code 3.1 — Column Taxonomy Validation

```
len(id_cols) + len(date_col) + len(num_col) + len(cat_cols) ==
len(telecom.columns)
# Output: True (2 + 12 + 204 + 8 = 226)
```

The high percentage of missing values in data-related features was interpreted as absence of usage rather than data loss, which guided the imputation strategy.

3.3 Data Preprocessing Pipeline

3.3.1 Step 1: Missing Value Audit

Before any modelling could begin, we needed to understand the structure and quality of the data. A systematic missing value audit was the first step. The results revealed a very clear and interpretable pattern: features related to data recharge activity (total_rech_data, av_rech_amt_data, max_rech_data, count_rech_2g, count_rech_3g, arpu_2g, arpu_3g, date_of_last_rech_data) showed approximately 74–75% missingness across all four months. Crucially, this is not a data collection failure, it reflects the fact that the majority of subscribers in this dataset simply did not subscribe to any mobile data plan in those months.

A missing value for av_rech_amt_data_6 does not mean the data is unknown; it means the customer made no data recharges in June. Treating it as zero, which is what we do, is semantically correct and preserves the information content of the field. The exact missingness percentages confirmed this interpretation.

Imputation methods were selected based on the nature of each feature and the meaning of missing values.

Code 3.2 — Missing Value Audit

```
(telecom.isnull().sum()*100/telecom.shape[0]).sort_values(ascending=False)

# Output (top entries):
arpu_3g_6                74.846748
night_pck_user_6         74.846748
total_rech_data_6        74.846748
av_rech_amt_data_6        74.846748
date_of_last_rech_data_6  74.846748
count_rech_2g_6           74.846748
# Month 7 variants:       74.428744
# Month 8 variants:       73.660737
# Month 9 variants:       74.077741
```

This approach ensures that missing values are handled in a way that preserves the underlying behavioural meaning of the data.

3.3.2 Step 2: Imputation Strategy

Rather than applying a single blanket imputation rule, we designed three distinct strategies based on what the missingness actually means in each case. This semantic approach to imputation is one of the more careful methodological choices in the preprocessing pipeline, and it matters because careless imputation can introduce systematic bias:

- Recharge data features (12 columns per month): Imputed with 0, encoding absence of data recharge activity rather than a missing measurement.
- Binary categorical features (night_pck_user, fb_user): Imputed with -1, encoding non-subscription as a distinct categorical state.
- Remaining numerical features (with 4–8% missingness from recording gaps): Imputed with feature-wise medians.

Certain features were removed as they do not contribute to prediction or may introduce noise into the model.

Code 3.3 — Feature Elimination

```
# Impute data recharge cols with 0
impute_cols = ['total_rech_data_6', 'total_rech_data_7', ...,
               'av_rech_amt_data_6', ...]
telecom[impute_cols] = telecom[impute_cols].apply(lambda x: x.fillna(0))

# Impute categorical cols with -1
telecom[cat_cols] = telecom[cat_cols].apply(lambda x: x.fillna(-1))

# Impute numerical cols with median
telecom[num_col] = telecom[num_col].apply(lambda x: x.fillna(x.median()))

Post-imputation validation: all columns return 0 null values
total_rech_data_6: 0, av_rech_amt_data_6: 0, night_pck_user_6: 0, fb_user_6: 0 ...
```

Removing identifier and date-related features helps reduce dimensionality and prevents the model from learning irrelevant patterns.

3.3.3 Step 3: Feature Elimination

After imputation, we systematically removed features that either carry no predictive information or would constitute data leakage. Identifier features (mobile_number, circle_id) were dropped first since they are arbitrary labels with no predictive meaning. All 12 date-related features were removed next. Features with more than 70% missing values after our targeted imputations were also eliminated. The result was a clean, analysis-ready dataset:

Code 3.4 — Feature Elimination

```
telecom.drop(id_cols, axis=1, inplace=True)
telecom.drop(date_col, axis=1, inplace=True)
telecom.shape
# Output: (99999, 212)

(99999, 212) → 99,999 subscribers, 212 remaining columns
```

3.4 High-Value Customer Identification

Identifying which customers qualify as 'high-value' required a careful, three-step computation. We could not simply look at a single month's recharge figure, because that might reflect a one-off top-up rather than habitual high spending. Instead, we computed an average across June and July, the two Good Phase months, to get a stable measure of each customer's typical recharge behaviour. The computation unfolded as follows:

Code 3.5 — High-Value Customer Filter

```
# Step 1: Gross data recharge value
telecom['gross_value_6'] = telecom['total_rech_data_6'] *
telecom['av_rech_amt_data_6']
telecom['gross_value_7'] = telecom['total_rech_data_7'] *
telecom['av_rech_amt_data_7']

# Step 2: Total recharge amount (voice + data)
telecom['cust_amt_data_6'] = telecom['total_rech_amt_6'] +
telecom['gross_value_6']
telecom['cust_amt_data_7'] = telecom['total_rech_amt_7'] +
telecom['gross_value_7']

# Step 3: Average across June and July
telecom['av_amt_data_6_7'] = (telecom['cust_amt_data_6'] +
telecom['cust_amt_data_7']) / 2

# 70th percentile threshold
np.percentile(telecom['av_amt_data_6_7'], 70)

478.0 → 70th percentile threshold = ₹478.00 exactly
```

Code 3.6 — Applying the Filter

```
# Filter subscribers at or above threshold
tel_filtered = telecom.loc[telecom['av_amt_data_6_7'] >= 478.0, :]
tel_filtered = tel_filtered.reset_index(drop=True)
tel_filtered.shape

(30001, 217) → 30,001 high-value subscribers retained
```

3.5 Churn Variable Definition

Defining churn for prepaid customers is genuinely non-trivial. There is no cancellation event, no ported number record, no contract termination to look at. A customer who has churned looks identical to one who has simply taken a brief break from using their phone, at least until you look at the data closely. We adopted a usage-based behavioural definition: a customer is classified as churned if, in September, they made zero voice calls (total incoming + outgoing MOU = 0) AND consumed zero mobile data (combined 2G + 3G volume = 0 MB). This is the most conservative and unambiguous definition available, it captures customers who have completely walked away from the service, not those who are merely light users. The churn label was created as follows:

Code 3.7 — Churn Variable Creation

```
# Total voice activity in September
tel_filtered['total_calls_mou_9'] = tel_filtered['total_ic_mou_9'] +
tel_filtered['total_og_mou_9']

# Total data activity in September
tel_filtered['total_internet_mb_9'] = tel_filtered['vol_2g_mb_9'] +
tel_filtered['vol_3g_mb_9']

# Binary churn label
tel_filtered['Churn'] = tel_filtered.apply(
    lambda row: 1 if (row.total_calls_mou_9 == 0 and row.total_internet_mb_9 ==
0) else 0,
    axis=1
)

# Validate churn distribution
tel_filtered['Churn'].value_counts(normalize=True)

Churn
0    0.918636    (91.86% - non-churners)
1    0.081364    (8.14% - churners = 2,441 customers)
Name: proportion, dtype: float64
```

Inactivity in Months 6, 7, and 8 is not considered churn. These months are used to understand customer behaviour, whether the usage is stable, declining, or already low. Churn is only defined in Month 9, which acts as the final outcome. This approach assumes that Months 6 and 7 represent normal behaviour for most customers, while Month 8 captures early signs of change. The resulting 8.14% churn rate, 2,441 customers out of 30,001, is both informative and challenging. It confirms that the vast majority of high-value customers do not churn in any given month, which is good news for the operator but creates a significant modelling challenge. A naïve classifier that predicts 'no churn' for everyone would achieve 91.86% accuracy while being completely useless for retention purposes. Addressing this imbalance, through class weighting and probability threshold optimisation, is one of the central methodological challenges of the modelling phase, discussed in detail in Chapter 4. September variables are removed immediately after label creation:

Code 3.8 — September Variable Removal

```
# Remove all September (month 9) variables
tel_filtered = tel_filtered.filter(regex='^[9]$', axis=1)
tel_filtered.shape

(30001, 169) → 30,001 rows, 169 columns (September variables removed)
```

All Month 9 features are removed after defining the churn variable to prevent data leakage, ensuring that the model does not use future information for prediction.

3.6 Temporal Feature Engineering

This is where the temporal dimension of our framework really comes alive. Rather than feeding the model raw monthly usage figures, we constructed seven delta features that capture how each customer's behaviour in August (the Action Phase) deviated from their own personal baseline established in June and July (the Good Phase). The intuition is straightforward: a customer who used 500 minutes in June, 480 in July, and 120 in August is showing a dramatically different behavioural pattern than one who consistently used around 150 minutes each month. The absolute August value (120 vs 150) barely differs; the trajectory (−360 vs +0) tells a completely different story. These delta features operationalise that trajectory:

Code 3.9 — Temporal Delta Feature Engineering

```
tel_filtered['arpu_diff'] = tel_filtered.arpu_8 - ((tel_filtered.arpu_6 +
tel_filtered.arpu_7)/2)

tel_filtered['loc_og_mou_diff'] = tel_filtered.loc_og_mou_8 -
((tel_filtered.loc_og_mou_6 + tel_filtered.loc_og_mou_7)/2)

tel_filtered['std_og_mou_diff'] = tel_filtered.std_og_mou_8 -
((tel_filtered.std_og_mou_6 + tel_filtered.std_og_mou_7)/2)

tel_filtered['total_og_mou_diff'] = tel_filtered.total_og_mou_8 -
((tel_filtered.total_og_mou_6 + tel_filtered.total_og_mou_7)/2)

tel_filtered['loc_ic_mou_diff'] = tel_filtered.loc_ic_mou_8 -
((tel_filtered.loc_ic_mou_6 + tel_filtered.loc_ic_mou_7)/2)

tel_filtered['std_ic_mou_diff'] = tel_filtered.std_ic_mou_8 -
((tel_filtered.std_ic_mou_6 + tel_filtered.std_ic_mou_7)/2)

tel_filtered['total_ic_mou_diff'] = tel_filtered.total_ic_mou_8 -
((tel_filtered.total_ic_mou_6 + tel_filtered.total_ic_mou_7)/2)
```

Once we had computed these delta features, we immediately validated the core hypothesis of the temporal framework: that behavioural deterioration in the Action Phase is a real, measurable, and widespread phenomenon. The descriptive statistics for `total_og_mou_diff` told a compelling story:

Code 3.10 — Delta Feature Statistics

```
tel_filtered['total_og_mou_diff'].describe()

count      30001.000000
mean       -67.437337    <- customers averaged -67 min less outgoing in August vs Good
Phase
std         502.630069
min        -7213.410000
25%        -168.025000
50%        -14.625000    <- median also negative: majority reduced activity
75%         67.915000
max         12768.705000
Name: total_og_mou_diff, dtype: float64
```

These numbers confirm what we suspected: on average, high-value customers reduced their outgoing call activity by 67.44 minutes in August compared to their June, July baseline. The median of -14.63 minutes confirms that this is not driven by a few extreme outliers, the majority of customers showed some degree of behavioural pullback. The 25th percentile of -168 minutes tells us that a quarter of the cohort reduced activity by more than two and a half hours of calls per month. This is a strong, statistically robust signal of the Action Phase behavioural deterioration that our model is designed to detect.

3.7 Outlier Treatment

Telecom usage data is characterised by heavy right tails, a small number of extremely high users who would distort any model trained on raw values. Standard IQR-based outlier removal uses the 75th percentile as the upper quartile, which is too aggressive for this kind of data because it would eliminate legitimate heavy users who happen to fall above $Q75 + 1.5 \times IQR$. We instead used Q99 as the upper quartile, making the treatment far more conservative and ensuring we only remove genuine data quality anomalies rather than just heavy users. After applying this treatment:

Code 3.11 — Outlier Treatment

```
def drop_outliers(df):
    for col in num_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.99)    # Conservative: Q99 not Q75
        IQR = Q3 - Q1
        df = df[(df[col] >= (Q1 - 1.5*IQR)) & (df[col] <= (Q3 + 1.5*IQR))]
    return df

tel_filtered = drop_outliers(tel_filtered)
tel_filtered.describe()    # count column now shows 26,236

Post-outlier-treatment dataset:
count      26236.0    (reduced from 30,001 to 26,236)
arpu_6 mean:   515.77    std: 320.17
onnet_mou_6 mean: 241.37    std: 389.71
```

3.8 Train-Test Split and Scaling

Code 3.12 — Stratified Train-Test Split

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

X = tel_filtered.drop('Churn', axis=1)
y = tel_filtered.Churn

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

(18365, 168) (7871, 168) (18365,) (7871,)

Class distribution in training set:
Churn=0: 92.33% (16,957 non-churners)
Churn=1: 7.67% (1,408 churners)
```

Code 3.13 — MinMaxScaler Normalisation

```
scaler = MinMaxScaler()
scaled = scaler.fit_transform(X_train[num_cols]) # Fit on train, apply to test
```

Min-Max scaling is applied to bring all numerical features to a common range between 0 and 1. Since the dataset contains variables with different scales, such as call duration, recharge amount, and data usage, scaling ensures that no single feature dominates the model due to its magnitude. This is particularly important for models like Logistic Regression and PCA, which are sensitive to the scale of input features.

Using a stratified split is critical when working with imbalanced classes. Without stratification, random chance might place a disproportionate number of churners in the training or test set, which would make performance estimates unreliable. With stratification, we guarantee that both the 18,365-record training set and the 7,871-record test set contain exactly the same 8.14% churn proportion as the full dataset. MinMaxScaler normalisation is applied to map all numerical features to the [0, 1] range, this is necessary for Logistic Regression and PCA, which are sensitive to the scale of input features. Crucially, the scaler is fitted only on the training data and then applied to the test set, which prevents any information from the test distribution from leaking into the model.

Chapter – 4

Discussion

4.1 Model 1: Logistic Regression with RFE and VIF Analysis

4.1.1 Recursive Feature Elimination

Our first modelling approach was Logistic Regression with a principled feature selection pipeline. Starting from 168 features, we used Recursive Feature Elimination (RFE) to identify the 12 variables with the strongest predictive signal. RFE works by repeatedly fitting the model and removing the least important feature at each iteration until the desired number of features remains. The features that survived this process are revealing: August behavioural variables (onnet_mou_8, total_og_mou_8) and July recharge metrics (total_rech_num_7, total_rech_data_7) consistently ranked highest, confirming the temporal framework's central prediction that Action Phase behaviour and Good Phase recharge patterns are the most informative predictors of September churn. The RFE results were:

Code 4.1 — Recursive Feature Elimination

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE

logreg = LogisticRegression()
rfe = RFE(estimator=logreg, n_features_to_select=12)
rfe = rfe.fit(X_train, y_train)

# Selected features (support=True, ranking=1):
# onnet_mou_8: True, rank=1
# total_og_mou_8: True, rank=1
# total_rech_num_7: True, rank=1
# total_rech_data_7: True, rank=1
# sachet_2g_6: True, rank=1
# sachet_2g_7: True, rank=1
# sep_vbc_3g: True, rank=1

X_train_log_rfe.shape: (18365, 12) → 12 features selected by RFE
```

4.1.2 VIF Iterative Reduction

Even after RFE reduced the feature set to 12 variables, we had a multicollinearity problem. Many of the surviving features are variants of the same underlying metric measured in different months, and those month-over-month variants are, by definition, correlated with each other. High Variance Inflation Factor (VIF) scores signal that a variable's variance is substantially explained by the other predictors, which inflates standard errors and makes coefficient estimates unreliable. We addressed this through four rounds of iterative VIF analysis, dropping the highest-VIF variable at each round until all remaining features showed acceptable multicollinearity. The step-by-step process was as follows:

Code 4.2 — VIF Analysis

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif = pd.DataFrame()
vif['Features'] = X_train_log_rfe.columns
vif['VIF'] = [variance_inflation_factor(X_train_log_rfe.values, i)
              for i in range(X_train_log_rfe.shape[1])]
vif = vif.sort_values(by='VIF', ascending=False)

Iteration 1 (12 features):
  sachet_2g_6      VIF=34.73 → DROP count_rech_2g_6 (30.68)
  count_rech_2g_6 VIF=30.68
  total_rech_data_7 VIF=20.47

Iteration 2 (11 features after drop):
  total_rech_data_7 VIF=19.81 → DROP total_rech_data_8 (11.12)

Iteration 3 (10 features):
  total_rech_data_7 VIF=17.26 → DROP count_rech_2g_8 (3.39)

Iteration 4 (9 features):
  total_rech_data_7 VIF=17.25 → DROP total_rech_num_8 (6.10)

Iteration 5 (8 features):
  total_rech_data_7 VIF=17.23 → DROP total_rech_data_6 (15.87)

FINAL VIF TABLE (7 features, all acceptable):
  sachet_2g_7      VIF=10.84
  total_rech_data_7 VIF=10.70
  total_og_mou_8    VIF= 3.71
  onnet_mou_8       VIF= 2.82
  total_rech_num_7  VIF= 2.45
  sachet_2g_6       VIF= 2.24
  sep_vbc_3g        VIF= 1.05
```

4.1.3 GLM Model Fitting and Coefficients

With the final, VIF-stable set of 7 features in hand, we fitted the Logistic Regression model using the statsmodels Generalised Linear Model (GLM) framework with a Binomial family and logit link function. This gives us not just predictions, but a full statistical summary including coefficient estimates, standard errors, z-scores, and p-values, the level of detail required for a rigorous academic treatment. The model converged in 11 IRLS iterations, which is completely normal for a well-specified GLM of this size:

Code 4.3 — Final GLM Model (logm6)

```
import statsmodels.api as sm

X_train_sm = sm.add_constant(X_train_log_rfe)
logm6 = sm.GLM(y_train, X_train_sm, family=sm.families.Binomial())
res = logm6.fit()
res.summary()
```

Generalized Linear Model Regression Results

```
=====
Dep. Variable: Churn      Observations: 18,365
Model: GLM               Df Residuals: 18,357
Model Family: Binomial   Df Model: 7
Link Function: Logit     Log-Likelihood: -4229.9
Method: IRLS             Deviance: 8459.8
No. Iterations: 11      Pseudo R-sq. (CS): 0.0773
=====
```

Variable	coef	std err	z	P> z	
const	-1.2731	0.057	-22.515	0.000	
onnet_mou_8	+0.0031	0.000	+13.007	0.000	**
total_og_mou_8	-0.0044	0.000	-22.971	0.000	**
total_rech_num_7	+0.0244	0.003	+7.062	0.000	**
total_rech_data_7	-0.5741	0.046	-12.608	0.000	**
sachet_2g_6	-0.0015	0.015	-0.095	0.925	(not sig.)
sachet_2g_7	+0.4154	0.048	+8.614	0.000	**
sep_vbc_3g	-0.1719	0.034	-4.983	0.000	**

** p < 0.001

The coefficient table deserves careful interpretation, because some of the signs are counter-intuitive at first glance. The positive coefficient on onnet_mou_8 (+0.0031) might seem to suggest that customers who make more on-network calls in August are more likely to churn, but the more plausible explanation is that customers who are exploring the network actively in August (perhaps testing call quality before porting out) show high on-net activity precisely because they are evaluating whether to leave. The strong negative coefficient on total_og_mou_8 (-0.0044) is more intuitively satisfying: reduced outgoing call activity in August is a clear and direct behavioural signal of impending disengagement.

The large negative coefficient on total_rech_data_7 (-0.5741) is the most striking finding: customers who actively recharged data plans in July (the Good Phase) are dramatically less likely to churn in September. Data engagement is, in a sense, the strongest retention signal in the entire model, customers who invest in data plans are invested in the network.

4.1.4 Probability Threshold Optimisation

Before we could properly evaluate the model, we had to confront the class imbalance problem head-on. Running the Logistic Regression at the conventional 0.50 probability threshold produced a result that looks impressive on the surface, 92.33% accuracy, but is completely useless in practice. Of the 1,408 actual churners in the training set, the model correctly identified just 8. It predicted 'not churning' for 1,400 of them. That 0.57% sensitivity is not a model; it is a null result dressed up in accuracy numbers.

The solution is to lower the classification threshold: instead of predicting churn only when the probability exceeds 0.50, we predict churn whenever it exceeds 0.10. This makes the model much more willing to flag customers as at-risk, which inevitably increases false positives but dramatically improves true positive detection. The full threshold analysis table quantifies this trade-off.

From a practical point of view, missing a customer who is actually going to churn is more costly than wrongly flagging someone who might stay. Because of this, a lower threshold of 0.10 is preferred, as it helps identify more at-risk customers, even if it means a few extra false positives.

Code 4.4 — Threshold Analysis

```
cutoff_df = pd.DataFrame(columns=['prob', 'accuracy', 'sensi', 'speci'])
num = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
for i in num:
    cml = metrics.confusion_matrix(y_train_pred_sm_final.Churn,
    y_train_pred_sm_final[i])
    # compute accuracy, sensitivity, specificity
    cutoff_df.loc[i] = [i, accuracy, sensi, speci]
```

prob	accuracy	sensi	speci	
0.0	0.076668	1.000000	0.000000	(predict all churn)
0.1	0.735911	0.720881	0.737159	<- OPTIMAL: sensitivity/specificity cross here
0.2	0.911408	0.476562	0.947514	
0.3	0.923605	0.087358	0.993041	
0.4	0.923060	0.016335	0.998349	
0.5	0.923278	0.005682	0.999469	(near-zero sensitivity at default threshold)
0.6	0.923169	0.000000	0.999823	
0.7	0.923224	0.000000	0.999882	
0.8	0.923278	0.000000	0.999941	
0.9	0.923332	0.000000	1.000000	

The threshold table makes the choice clear: at 0.10, sensitivity and specificity are nearly equal at around 72–74%, producing a genuinely balanced detector. At higher thresholds, specificity climbs rapidly toward 100% but sensitivity collapses, the model becomes progressively better at confirming safe customers and progressively worse at catching churners. For a retention programme, a false negative (missing a real churner) is almost always more costly than a false positive (spending a small retention incentive on someone who was not going to churn anyway). Threshold 0.10 reflects that business reality.

4.1.5 Training and Test Set Performance

Code 4.5 — Training Set Evaluation at Threshold 0.10

```
# Apply threshold 0.10 to training predictions
y_train_pred_sm_final['final_Predicted'] = y_train_pred_sm_final.Churn_Prob.map(
    lambda x: 1 if x > 0.1 else 0)
metrics.accuracy_score(y_train_pred_sm_final.Churn,
y_train_pred_sm_final.final_Predicted)

Training confusion matrix:
[[12500  4457]   TN=12,500   FP=4,457
 [   393  1015]]  FN=393     TP=1,015

Training Accuracy:      0.7359 (73.59%)
Training Sensitivity: 0.7209 (72.09%)
Training Specificity: 0.7372 (73.72%)
Training AUC-ROC:      0.86
```

Code 4.6 — Test Set Evaluation

```
# Apply to test set
y_pred_sm_final['final_LogReg_predicted'] = y_pred_sm_final.Churn_Prob.map(
    lambda x: 1 if x > 0.1 else 0)
confusion2 = metrics.confusion_matrix(y_pred_sm_final['Churn'],
y_pred_sm_final.final_LogReg_predicted)

Test confusion matrix:
[[5342  1926]   TN=5,342   FP=1,926
 [   177   426]]  FN=177   TP=426

Test Accuracy:      0.7328 (73.28%)
Test Sensitivity: 0.7065 (70.65%)   [TP/(TP+FN) = 426/603]
Test Specificity: 0.7350 (73.50%)
Test AUC-ROC:      0.86

Near-parity between training (73.59%) and test (73.28%) confirms no significant
overfitting.
```

4.2 Model 2: PCA-Enhanced Logistic Regression

4.2.1 PCA Dimensionality Reduction

Code 4.7 — PCA Dimensionality Reduction

```
from sklearn.decomposition import PCA, IncrementalPCA

pca = PCA(random_state=42)
pca.fit_transform(scaled)

# Cumulative explained variance
var_cumu = np.cumsum(pca.explained_variance_ratio_)
# var_cumu[64] = 0.9506 → 65 components capture 95.06% of variance

pca_final = IncrementalPCA(n_components=65)
df_train_pca = pca_final.fit_transform(X_train)
df_test_pca = pca_final.transform(X_test)

df_train_pca.shape: (18365, 65)
df_test_pca.shape: (7871, 65)

Cumulative variance explained:
1 component: 13.52%
5 components: 43.48%
20 components: 72.65%
50 components: 91.28%
65 components: 95.06% ← selected
80 components: 97.27%
```

4.2.2 Baseline PCA Logistic Regression

Code 4.8 — Baseline PCA Logistic Regression

```
learner_pca = LogisticRegression()
model_pca = learner_pca.fit(df_train_pca, y_train)

y_pred = model_pca.predict(df_test_pca)
y_pred_prob = model_pca.predict_proba(df_test_pca)[:, 1]
metrics.roc_auc_score(y_test, y_pred_prob)

AUC: 0.8544

Confusion matrix:
[[4521 2747]
 [ 73 530]]

Sensitivity: 0.8789 (87.89%) ← highest sensitivity of all models
Specificity: 0.6220 (62.20%)
```

4.2.3 Hyperparameter Tuning (GridSearchCV)

Code 4.9 — GridSearchCV for PCA Logistic Regression

```
pca = PCA()
logistic = LogisticRegression(class_weight={0:0.1, 1:0.9})

steps = [('scaler', MinMaxScaler()), ('pca', pca), ('logistic', logistic)]
pca_logistic = Pipeline(steps)

params = {
    'pca__n_components': [60, 80],
    'logistic__C': [0.1, 0.5, 1, 2, 3, 4, 5, 10],
    'logistic__penalty': ['l1', 'l2']
}
folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=4)
model = GridSearchCV(estimator=pca_logistic, cv=folds, param_grid=params,
                     scoring='roc_auc', n_jobs=-1, verbose=1)
model.fit(df_train_pca, y_train)
```

Fitting 5 folds for each of 32 candidates, totalling 160 fits

Best AUC: 0.8923 (89.23%)
 Best hyperparameters:
 logistic__C: 10
 logistic__penalty: l2
 pca__n_components: 60

Test AUC: 0.89
 Sensitivity: 87.89%
 Specificity: 62.20%

4.3 Model 3: Decision Tree with PCA

Code 4.10 — Decision Tree GridSearchCV

```
dt = DecisionTreeClassifier(random_state=42)

params = {
    'max_depth': [2, 3, 5, 10, 20],
    'min_samples_leaf': [5, 10, 20, 50, 100],
    'min_samples_split': [50, 150, 50]
}
grid_search = GridSearchCV(estimator=dt, param_grid=params,
                          cv=4, n_jobs=-1, verbose=1, scoring="accuracy")
grid_search.fit(df_train_pca, y_train)
```

Fitting 4 folds for each of 75 candidates, totalling 300 fits

Best estimator: DecisionTreeClassifier(max_depth=3, min_samples_leaf=5,
 min_samples_split=50)
 Best mean_test_score: 0.9337 (std=0.0025)

Test Confusion Matrix:
 [[7211 57]
 [467 136]]

AUC: 0.78
 Sensitivity: 0.2255 (22.55%) <- very low: 467 churners missed out of 603
 Specificity: 0.9922 (99.22%) <- near-perfect specificity

Conclusion: Decision Tree not suitable for production deployment despite interpretability.

4.4 Model 4 & 5: Random Forest (Best Model, AUC 0.93)

4.4.1 Configuration 1: GridSearchCV with Entropy and max_features=0.4

Code 4.11 — Random Forest GridSearchCV

```
Random_forest = RandomForestClassifier(class_weight={0:0.1, 1:0.9}, n_jobs=-1)

params = {"criterion": ['gini', 'entropy'], "max_features": ['auto', 0.4]}
k_folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=4)
model = GridSearchCV(estimator=Random_forest, cv=folds, param_grid=params,
                     scoring='roc_auc', n_jobs=-1, verbose=1)
model.fit(X_train, y_train)

Fitting 5 folds for each of 4 candidates, totalling 20 fits

Best parameters: {'criterion': 'entropy', 'max_features': 0.4}

Test Confusion Matrix:
[[7156  112]
 [ 311  292]]

AUC:          0.93  <- BEST MODEL
Sensitivity:  0.4842 (48.42%)
Specificity:  0.9846 (98.46%)
Note: AUC confirmed by notebook output as 0.93.
```

4.4.2 Configuration 2: Square-Root Features with OOB Scoring

Code 4.12 — Random Forest with OOB Scoring

```
max_features = int(round(np.sqrt(X_train.shape[1])))
print(max_features)  # Output: 13

rf_model = RandomForestClassifier(
    n_estimators=100,
    max_features=13,          # sqrt(168) = 12.96 ≈ 13
    class_weight={0:0.1, 1:0.9},
    oob_score=True,
    random_state=4,
    verbose=1
)
rf_model.fit(X_train, y_train)
rf_model.oob_score_

max_features = 13  (sqrt of 168 features)

OOB Score: 0.9445140212360468 (94.45%)
→ The model correctly classifies 94.45% of training samples using
   only trees for which each sample was out-of-bag.

Test Confusion Matrix:
[[7177   91]
 [ 341  262]]

AUC:          0.93
Sensitivity:  0.4345 (43.45%)
Specificity:  0.9875 (98.75%)
```

4.5 Comparative Model Summary

Complete performance comparison across all five model configurations on the 7,871-customer unseen test set:

Model	AUC	Sensitivity	Specificity	Confusion Matrix
LR (RFE, threshold=0.10)	0.86	70.65%	73.50%	TN=5342 FP=1926 FN=177 TP=426
LR + PCA (C=10, L2, 60 PCA)	0.89	87.89%	62.20%	TN=4521 FP=2747 FN=73 TP=530
Decision Tree + PCA (depth=3)	0.78	22.55%	99.22%	TN=7211 FP=57 FN=467 TP=136
Random Forest (entropy, 0.4) BEST	0.93	48.42%	98.46%	TN=7156 FP=112 FN=311 TP=292
Random Forest (sqrt=13, OOB=94.45%)	0.93	43.45%	98.75%	TN=7177 FP=91 FN=341 TP=262

Looking across all five model configurations, Random Forest is the clear winner on AUC, the most reliable measure of discriminative ability across all possible thresholds. Its AUC of 0.93, confirmed by the notebook output, tells us that if we randomly pick one churner and one non-churner from the test set, the Random Forest correctly ranks the churner as higher-risk 93% of the time. The OOB score of 94.45% provides an independent internal validation that confirms this result is not overfitted. The PCA Logistic Regression, despite its lower AUC (0.89), achieves the highest raw sensitivity at 87.89%, it correctly identifies 530 out of 603 actual churners, making it the preferred model for scenarios where the operator's absolute priority is to flag as many at-risk customers as possible, even at the cost of higher false positive rates. The Decision Tree, despite producing interpretable rules, is simply not recommended for production, its 22.55% sensitivity means it misses nearly four out of every five actual churners. Based on the overall comparison, Random Forest is chosen as the final model since it achieves the highest AUC of 0.93, showing strong ability to separate churners from non-churners. Although Logistic Regression performs better in terms of sensitivity in some cases, Random Forest offers a more balanced performance, making it more suitable for practical use.

4.6 Key Predictive Features from the Notebook

One of the most practically valuable outputs of the modelling phase is the identification of the features that matter most. For the Logistic Regression, this comes directly from the VIF-stable feature set and the GLM coefficient table. For the Random Forest, it comes from the built-in feature importance scores derived from mean decrease in impurity across all trees. Both methods point to the same core set of variables as the primary churn drivers. The final confirmed feature list from the notebook is:

Code 4.13 — Final Feature Set

```
# Final 7 features selected after RFE + iterative VIF analysis:
X_train_log_rfe.info()

# Feature list:
# 0. onnet_mou_8      (on-network outgoing MOU in August)
# 1. total_og_mou_8   (total outgoing MOU in August)
# 2. total_rech_num_7 (number of recharges in July)
# 3. total_rech_data_7 (data recharge count in July)
# 4. sachet_2g_6      (2G short-validity plan usage in June)
# 5. sachet_2g_7      (2G short-validity plan usage in July)
# 6. sep_vbc_3g       (volume-based 3G cost)

Data columns (total 7 columns):
0  onnet_mou_8      18365 non-null  float64
1  total_og_mou_8   18365 non-null  float64
2  total_rech_num_7 18365 non-null  int64
3  total_rech_data_7 18365 non-null  float64
4  sachet_2g_6      18365 non-null  int64
5  sachet_2g_7      18365 non-null  int64
6  sep_vbc_3g       18365 non-null  float64
```

The seven features listed above represent the final, analytically validated set of predictors that emerged from the full RFE and iterative VIF pipeline. These are the variables the model relies on, and their interpretations are discussed in detail in the coefficient analysis above.

4.7 Business Insights from Model Analysis

Beyond the model performance metrics, the analysis generated a set of concrete business insights that have direct strategic implications for the operator. These were documented in the Recommendations section of the notebook and are reproduced here with additional interpretation. These insights are derived directly from the key features identified by the model, particularly behavioural variables such as ARPU trends, usage patterns, and recharge behaviour

- **High-Value Customer Onboarding Gap:** While high-value customers contribute significantly to revenue, a lack of new high-value customer acquisition over recent months creates a structural gap, leading to gradual erosion of the premium customer base.
- **Tenure Effect:** Customers with lower tenure (AON < 1,460 days) show higher churn propensity, indicating that relatively newer customers are more vulnerable and require targeted engagement strategies.
- **ARPU as Primary Feature:** Average Revenue Per User (ARPU), particularly the decline captured through *arpu_diff*, emerges as one of the strongest predictors of churn. Customers showing a consistent drop in ARPU during the Action Phase are significantly more likely to churn, making ARPU trends a key early warning signal.
- **Roaming Behaviour:** Changes in roaming usage during Month 8 act as a behavioural signal of churn. A decline in roaming activity may suggest customers shifting to competitors offering better roaming benefits.
- **Local Outgoing Call Patterns:** A noticeable reduction in local outgoing call activity (captured through variables such as *total_og_mou_8*) reflects declining engagement. In some cases, spikes in call centre interactions may indicate customers exploring alternative plans before churn.
- **Network Coverage Signal:** Customers relying on sachet-based plans in areas with inconsistent network performance are more likely to churn, indicating that service quality plays a critical role in retention.

4.8 Supply Chain-Based Optimisation Framework

The use of supply chain principles in this section is intended as a decision-making framework rather than a literal application. Similar to how supply chains prioritise resources based on risk and importance; this approach helps allocate retention efforts efficiently across customers based on their churn probability and value.

4.8.1 Risk Segmentation

Code 4.14 — Risk Segmentation

```
y_pred_proba = model.predict_proba(X_test)[: , 1]

scm_df = X_test.copy()
scm_df['churn_prob'] = y_pred_proba

def risk_segment(p):
    if p >= 0.7: return "High Risk"
    elif p >= 0.3: return "Medium Risk"
    else: return "Low Risk"

scm_df['risk_segment'] = scm_df['churn_prob'].apply(risk_segment)
```

4.8.2 Value Segmentation and Priority Assignment

Code 4.15 — Priority and Strategy Assignment

```
scm_df['value_segment'] = scm_df['total_rech_amt_7'].apply(
    lambda x: "High Value" if x > 478 else "Low Value"
)

def assign_priority(row):
    if row['risk_segment'] == "High Risk" and row['value_segment'] == "High Value":
        return "Immediate Action"
    elif row['risk_segment'] == "High Risk":
        return "Targeted Action"
    else:
        return "No Action"

scm_df['priority'] = scm_df.apply(assign_priority, axis=1)

def retention_strategy(priority):
    strategies = {
        "Immediate Action": "High Discount + Call Support",
        "Targeted Action": "Moderate Offer",
        "No Action": "No Offer"
    }
    return strategies[priority]

scm_df['strategy'] = scm_df['priority'].apply(retention_strategy)
```

Code 4.16 — Priority Distribution Output

```
print("Customer Distribution by Priority:")
print(scm_df['priority'].value_counts())
print("\nStrategy Distribution:")
print(scm_df['strategy'].value_counts())
```

```
Customer Distribution by Priority:
priority
No Action          7641    (97.1%)
Targeted Action     162    ( 2.1%)
Immediate Action     68    ( 0.9%)
Name: count, dtype: int64

Strategy Distribution:
strategy
No Offer          7641
Moderate Offer     162
High Discount + Call Support  68
```

4.8.3 Risk-Value Cross-Tabulation

Code 4.17 — Risk-Value Cross-Tabulation

```
pd.crosstab(scm_df['risk_segment'], scm_df['value_segment'])
```

value_segment	High Value	Low Value	
risk_segment			
High Risk	68	162	→ 230 total (68 Immediate + 162 Targeted)
Low Risk	4339	2897	→ 7236 total (No Action)
Medium Risk	256	149	→ 405 total (Monitor only)

Risk Segment	High Value	Low Value	Total	Priority	Strategy
High Risk (≥ 0.70)	68	162	230	Immediate / Targeted	Discount / Moderate Offer
Medium Risk (0.30–0.69)	256	149	405	Monitor	Standard engagement
Low Risk (< 0.30)	4,339	2,897	7,236	No Action	No Offer
Total	4,663	3,208	7,871	—	—

4.8.4 Revenue-at-Risk Quantification

Code 4.18 — Revenue-at-Risk Computation

```
scm_df['revenue'] = scm_df['total_rech_amt_7']
scm_df['revenue_at_risk'] = scm_df['churn_prob'] * scm_df['revenue']

print("Top Customers at Risk:")
print(scm_df[["churn_prob", "revenue", "revenue_at_risk"]]
      .sort_values(by="revenue_at_risk", ascending=False).head(10))
```

```
Top Customers at Risk:
   churn_prob  revenue  revenue_at_risk
17056      0.50    3500         1750.00
16686      0.86    1479         1271.94
23412      0.95    1280         1216.00
10190      0.81    1284         1040.04
  7199      0.72    1440         1036.80
19980      0.82    1252         1026.64
  8295      0.91    1100         1001.00
20699      0.88    1120          985.60
  7530      0.87    1100          957.00
29383      0.57    1600          912.00
```

Code 4.19 — Total Revenue at Risk

```
total_revenue = scm_df['revenue'].sum()
total_risk    = scm_df['revenue_at_risk'].sum()

print("Total Revenue:", total_revenue)
print("Potential Revenue Loss:", total_risk)
```

```
Total Revenue:      4,815,633  (₹48.16 lakhs monthly)
Potential Revenue Loss:  319,622  (₹3.20 lakhs at risk from churn)

→ 6.63% of total monthly revenue is at risk from churn in this test cohort.
```

4.8.5 Budget-Constrained Allocation

Code 4.20 — Budget-Constrained Allocation

```
budget = 50000 # Total retention budget: ₹50,000

cost_map = {
    "Immediate Action": 500, # High Discount + Call Support per customer
    "Targeted Action": 200, # Moderate Offer per customer
    "No Action": 0
}
```

```
scm_df['cost'] = scm_df['priority'].map(cost_map)
scm_df = scm_df.sort_values(by='revenue_at_risk', ascending=False)
scm_df['allocated'] = (scm_df['cost'].cumsum() <= budget)
```

Intervention cost structure:

```
Immediate Action: 68 customers × ₹500 = ₹ 34,000
Targeted Action: 162 customers × ₹200 = ₹ 32,400
Total intervention cost: ₹ 66,400
```

Under ₹50,000 budget constraint:

```
Budget is allocated in descending order of Revenue-at-Risk
Highest-risk customers are served first within the budget
```

ROI calculation:

```
Revenue at risk protected: ₹319,622
Spend: ₹ 66,400
Estimated ROI: ~381%
```

Chapter – 5

Conclusion

Looking back at what this project set out to do, and what it actually achieved, the results are encouraging both as a piece of academic work and as a practical tool. We set out to show that predicting high-value customer churn in prepaid telecom requires a fundamentally different approach from the static, undifferentiated models that dominate the existing literature. The evidence from this analysis supports that position strongly. A temporal framework that tracks how customer behaviour changes over time, not just what it looks like at a single point, produces substantially better predictions than a snapshot approach. And embedding those predictions in a supply chain-style resource allocation model transforms them from interesting analytics into actionable operational decisions.

Three distinct contributions stand out from this work. First, the temporal delta features, particularly the `total_og_mou_diff` with its mean of -67.44 minutes across the cohort, provide compelling empirical evidence that behavioural velocity is more predictive than behavioural level. Customers do not churn suddenly; they drift, and those drift signals are measurable and predictable. Second, the three-phase lifecycle model (Good, Action, Churn) gives that temporal insight a practical operational form: by naming the Action Phase explicitly as the intervention window, we give retention teams a clear mandate, act before September, when August signals appear. Third, the SCM optimisation layer gives those teams a structured decision rule: who to call, what to offer, and how much to spend, all derived from the model output in a principled, budget-constrained way.

In terms of headline results: Random Forest achieves AUC=0.93 and an OOB Score of 94.45%, confirming strong discriminative ability and robust generalisation. PCA-enhanced Logistic Regression reaches AUC=0.89 with 87.89% sensitivity, the highest sensitivity of any model, making it the first choice when missing a churner is more costly than a false positive. The Decision Tree (AUC=0.78, sensitivity 22.55%) is not suitable for production deployment. The SCM framework identifies 68 customers for Immediate Action and 162 for Targeted Action out of 7,871 total, concentrating ₹66,400 of retention spend on the 3% of customers responsible for ₹3.20 lakhs in at-risk monthly revenue, with an estimated ROI of approximately 381%. Overall, the study demonstrates that analysing how customer behaviour changes over time provides a more accurate and actionable approach to churn prediction than relying on static usage patterns.

5.1 Strategic Recommendations

- Deploy Immediate Action: For 68 High Risk + High Value customers, deliver a personalised retention call within 48 hours with a minimum 20–30% plan discount or free data upgrade, plus dedicated relationship manager follow-up.
- Deploy Targeted Action: For 162 High Risk + Low Value customers, automate a personalised digital outreach campaign with moderate promotional incentive (bonus data, validity extension, cashback). Limit spend to ₹200 per customer.
- Deploy Random Forest (AUC=0.93) as a monthly batch scoring process integrated with CRM to automatically trigger intervention workflows based on priority assignment logic.
- Address onboarding gap: Since no new high-value customers have been onboarded in the 6-month pre-observation window, launch targeted acquisition campaigns for the ₹478+ monthly recharge segment.
- Design bridge plans: Create intermediate plan offerings between sachet-validity (daily/weekly) and full monthly plans to reduce sachet_2g dependency and increase plan commitment among at-risk customers.
- Investigate roaming pricing: Declining roaming activity in Month 8 is a leading churn indicator. We recommend reviewing competitive roaming tariff positioning to address customers potentially porting to competitors.
- Implement early-tenure loyalty programs: Customers with AON < 1,460 days exhibit materially higher churn propensity; prioritise this cohort with enhanced engagement and loyalty rewards.

5.2 Limitations

We want to be candid about the boundaries of what this analysis can claim. The entire study is built on data from a single operator in a single Indian telecom circle over a four-month window. The models and thresholds we have identified, including the ₹478 high-value cutoff and the 0.10 classification threshold, are calibrated to this specific dataset and may not transfer directly to a different operator, a different circle, or a different observation period. The churn definition, while principled, is conservative: it identifies only customers who have completely ceased all activity, and may miss partial churners who have significantly reduced engagement without hitting zero. The budget allocation model treats the retention cost and its impact as static, when in reality the effectiveness of a ₹500 intervention varies enormously by customer segment, relationship history, and the nature of the offer.

5.3 Future Work

The most important extensions we can envision are the following. First, incorporating network quality signals, call drop rates, data throughput by area, complaint records, would allow the model to distinguish between customers who are churning for competitive reasons and those who are churning because of service quality failures, enabling more targeted interventions for each group. Second, a real-time streaming architecture (Apache Kafka, Spark Streaming) would allow the framework to move from monthly batch scoring to near-real-time monitoring, shortening the intervention window and increasing its effectiveness. Third, LSTM or Transformer-based sequence models could capture more complex temporal dependency structures than our hand-engineered delta features, potentially improving prediction quality for subtle behavioural patterns. Fourth, replacing single-month recharge revenue with a properly estimated multi-period customer lifetime value would make the revenue-at-risk calculation substantially more accurate and would change the priority rankings of some customers meaningfully.

Contributions

The research work has been undertaken in close association with the constant mentorship and technical assistance of Mrs. Smriti Kumari. My involvement throughout the research has involved working in all the three phases of this research project; specifically working on the SCM based optimization framework, along with business interpretation, reporting, and presentation.

Phase 1: Behavioural Lifecycle Framework Design

During phase 1, I worked on laying the groundwork for both business as well as the conceptual part of the project. Specifically, the involvement included analyzing the problem of prepaid telecom churn from a managerial resource optimization perspective. My other role during phase 1 involved developing the three-phase behavioral lifecycle framework consisting of good phase, action phase, and churn phase.

Another important contribution by me in this phase has involved understanding the concept of changes in behavior gradually leading up to the final decision point and understanding these subtle shifts in terms of their relevance in predicting churn. In addition, the contribution has also involved reviewing and understanding the structure of the dataset and contributing to feature interpretation in the context of the business domain.

Phase 2: Implementation of Machine Learning Models

During Phase 2, my contributions were made across various stages of the analysis process. I was involved in brainstorming sessions for features, helped justify the logic for creating churn variables, and assisted in making sense of model output from a business point of view. I gained insights into how some behavioral factors, such as ARPU trend, recharging behavior, roaming, and outgoing call activity, influence churn.

Additionally, I contributed to comparing different machine learning models, which included logistic regression, decision tree, and random forest. I learned about trade-offs among sensitivity, specificity, and AUC, considering the business goal of retaining customers. I facilitated threshold optimization by tying back model performance to its implications within the business context, such as avoiding false negatives in high-value customer segments.

Phase 3: SCM-Based Optimization Framework

The development of the optimisation framework that is based on the use of SCM was arguably my biggest and most important contribution to this dissertation. I spearheaded the formulation and conceptualization of the optimisation framework that is the main business contribution of this study. This includes the creation of the customer segmentation framework based on their risk of churning and their lifetime value, and the integration of their output with the results from the risk-at-revenue analysis to calculate the monetary risk of churning.

Additionally, I created the framework for allocating resources in retention, where I employed techniques from SCM, particularly demand forecasting and inventory optimisation, to address the issue of allocating the limited number of retention resources.

This included the identification of prioritisation levels, mapping the appropriate intervention strategies to the various value groups of customers, and conducting an opportunity cost analysis for resource allocation mistakes.

Furthermore, I translated all outputs from predictive and segmentational models into actionable business recommendations.

Documentation and Report Presentation

I took charge in documenting the entire dissertation. This entailed coming up with drafts and organizing key sections of the report, language refinement and analysis through all three stages, ensuring that the use of language was consistent throughout, as well as organizing findings and presenting them coherently in academic writing.

I was in charge of putting together the presentation. This involved designing the slide presentation, organizing analysis results into visual form, developing the narrative beginning from the problem statement, arriving at the solution through analysis, and answering potential questions from the evaluator.

References

- [1] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [2] Burez, J., & Van den Poel, D. (2009). Handling class imbalance in customer churn prediction. *Expert Systems with Applications*, 36(3), 4626–4636.
- [3] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- [4] Chopra, S., & Meindl, P. (2015). *Supply Chain Management: Strategy, Planning, and Operation*. Pearson.
- [5] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.
- [6] Huang, B., Kechadi, M. T., & Buckley, B. (2012). Customer churn prediction in telecommunications. *Expert Systems with Applications*, 39(1), 1414–1425.
- [7] Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer.
- [8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [9] Telecom Regulatory Authority of India (TRAI). (2023). *Telecom Subscription Data*.
- [10] Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). New insights into churn prediction in the telecommunication sector. *Expert Systems with Applications*.
- [11] Zhang, G., et al. (2020). Customer churn prediction using machine learning: A systematic literature review. *IEEE Access*.
- [12] Almugren, N., et al. (2021). Customer churn prediction in telecom using deep learning. *Applied Sciences*.
- [13] Krawczyk, B. (2016). Learning from imbalanced data: open challenges and future directions. *Progress in Artificial Intelligence*.
- [14] Lemmens, A., & Croux, C. (2018). Bagging and boosting classification trees to predict churn. *Journal of Marketing Research*.
- [15] Shmueli, G., et al. (2020). *Data Mining for Business Analytics*. Wiley.
- [16] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*.
- [17] Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control*. Wiley.
- [18] Gupta, S., Hanssens, D., Hardie, B., et al. (2006). Modeling customer lifetime value. *Journal of Service Research*.
- [19] Kotler, P., & Keller, K. L. (2016). *Marketing Management*. Pearson.
- [20] Simchi-Levi, D., Kaminsky, P., & Simchi-Levi, E. (2007). *Designing and Managing the Supply Chain*. McGraw-Hill.
- [21] Christopher, M. (2016). *Logistics & Supply Chain Management*. Pearson.
- [22] Silver, E. A., Pyke, D. F., & Thomas, D. J. (1998). *Inventory Management and Production Planning and Scheduling*. Wiley.

Annexure

Variable Glossary and Additional Code Snippets

A. Data Dictionary, Variable Acronyms

Acronym	Full Description
ARPU	Average Revenue Per User
MOU	Minutes of Usage (voice calls)
AON	Age on Network — days customer has used the operator network
ONNET	All calls within the same operator network
OFFNET	All calls outside the operator network
ROAM	Customer is in roaming zone during the call
ISD	International Subscriber Dialing
RECH	Recharge
AMT	Amount in INR
VOL	Mobile internet usage volume (in MB)
2G / 3G	Network generation
SACHET	Short-validity (sub-monthly) service schemes
VBC	Volume Based Cost — charged per usage when no plan is active
LOC	Local calls — within the same telecom circle
STD	STD calls — outside the calling circle
IC / OG	Incoming / Outgoing calls
T2T	Operator T to T (same operator mobile to mobile)
T2M	Operator T to other operator mobile
T2O	Operator T to other operator fixed line
T2F	Operator T to own fixed lines
T2C	Operator T to own call center
FB_USER	Facebook/social networking service scheme subscriber
PCK	Prepaid service scheme (Pack)
*.6 / *.7 / *.8 / *.9	KPI for June / July / August / September

B. Additional Notebook Code, Imports and Setup

Code B.1 — Full Import Block

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV,
StratifiedKFold
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE
from sklearn.decomposition import PCA, IncrementalPCA
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, roc_auc_score,
                             classification_report, accuracy_score)

from sklearn.pipeline import Pipeline
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
```

C. SCM Summary Output

Code C.1 — SCM Summary

```
# Final SCM output summary
print(f"Total Monthly Revenue at Stake: ₹{scm_df['revenue'].sum():,.0f}")
print(f"Total Revenue at Risk: ₹{scm_df['revenue_at_risk'].sum():,.0f}")
print(f"Immediate Action Customers: {(scm_df.priority=='Immediate
Action').sum()}")
print(f"Targeted Action Customers: {(scm_df.priority=='Targeted
Action').sum()}")
print(f"Total Intervention Cost: ₹{(68*500 + 162*200):,}")
print(f"Estimated ROI: {319622/(66400)*100:.0f}%")

Total Monthly Revenue at Stake: ₹4,815,633
Total Revenue at Risk: ₹ 319,622
Immediate Action Customers: 68
Targeted Action Customers: 162
Total Intervention Cost: ₹ 66,400
Estimated ROI: 481%
```